Viva Guide: Distributed Food Ordering

Kafka 3.9.2 (KRaft) + Spark batch · Big Data 2026 (UE24CS343AB2) · Member 1 = VM1, Member 2 = VM2, Member 3 = VM3, Member 4 = VM4

How to use this guide. The viva is **individual**, so everyone must be able to explain the whole project, not just their own part. **Everyone** reads Parts 1, 2, 5 and 6 and the numbers in Part 7. **Your own member section** in Part 3 is where you'll get the deepest questions. Part 4 covers questions about the outputs the TA sees during the demo. Practise saying each answer out loud in one or two sentences.

Part	What's in it
1. The project in 2 minutes	What it does, the 4 VMs, the data flow
2. Reading the demo screen	Every checkpoint's output, line by line, and what it means
3. Member sections	Each member's code and role, with the questions most likely to be asked
4. Questions about the outputs	Why 62, 48, 44; why 16/28; why Isr 1; the CSV numbers
5. Concept questions	Kafka, KRaft, replication, producers, consumers, Spark, networking
6. Tricky and what-if questions	The follow-ups examiners like to ask
7. Numbers to remember	One table to revise just before the viva

Ten questions you are almost certain to get

- Explain the whole pipeline from `order.jsonl` to the two CSVs.
- What is KRaft and what does the controller do?
- What do Leader, Replicas and Isr mean in the describe output?
- What happens, step by step, when VM3 is killed? Why does the producer keep working?
- Why must VM2 never be stopped?
- Why did 62 records become 48, and then 44?
- Why does `order_metrics` always split 16 / 28 but `orders` doesn't?
- Why is replication factor 2 and not 3? What does min.insync.replicas=1 do?
- Why doesn't Spark read Kafka directly?
- How did you handle orders that cross midnight?

Part 1. The project in 2 minutes

A food delivery app produces a file of **62 raw orders** (`order.jsonl`). Some are junk: broken records, cancelled orders, bad timestamps and duplicates. The project moves this data between machines with **Kafka**, cleans and enriches it with **Spark**, sends it back through Kafka, and summarises it with Spark again into two reports: one per restaurant and one per city. It also proves that Kafka keeps working when one of its two servers (VM3) crashes.

The one-sentence version: Kafka moves the data reliably between independent stages; Spark does the batch computation at each end, always on a local file.

The four machines

VM	Role	Runs
VM1 (Member 1)	Producer 1, Consumer 2, Spark Job 2	`producer1.py` (raw orders in), `consumer2.py`, `spark_job2.py` (final CSVs)
VM2 (Member 2)	Kafka broker 1 + **KRaft controller**	The cluster's 'boss': knows which brokers are alive and who leads each partition
VM3 (Member 3)	Kafka broker 2	The second copy of every partition; the machine that is killed in the test
VM4 (Member 4)	Consumer 1, Spark Job 1, Producer 2	`consumer1.py`, `spark_job1.py` (cleaning + metrics), `producer2.py`

The data flow

```
VM1  producer1.py --[topic: orders, 62]--> VM4  consumer1.py -> orders_landed.jsonl (62)
                                                spark_job1.py -> order_metrics.jsonl (44)
VM1  consumer2.py <--[topic: order_metrics, 44]-- VM4  producer2.py
     -> order_metrics_landed.jsonl (44) -> spark_job2.py -> restaurant_summary.csv (6 rows)
                                                         -> city_summary.csv       (4 rows)

Kafka cluster = VM2 (controller + broker 1) + VM3 (broker 2), connected over ZeroTier
Both topics: 2 partitions, replication factor 2, min.insync.replicas = 1
```

Rules from the spec (and how we meet them)

Spec rule	How the project does it
Spark must not consume Kafka directly	A consumer writes a local .jsonl file first; Spark only reads files
No HDFS	Everything is on each machine's local disk (`outputs/`)
Job 1's output must travel through Kafka	Producer 2 publishes it to `order_metrics`; VM1 consumes it
2 partitions, RF 2, same cluster	`kafka/topics.sh create` makes both topics that way
Failure test stops VM3, not VM2	VM2 is the only controller; stopping it would stop the whole cluster
Exact duplicates counted once, in both jobs	`dropDuplicates` over all columns in Job 1 and Job 2
Round to 2 decimals	`F.round(..., 2)` on every calculated value
Nothing hardcoded	Every result is computed from the file; brokers and paths come from flags / env vars

Part 2. Reading the demo screen, checkpoint by checkpoint

The TA evaluates 4 checkpoints. For each one, here is what appears on screen and what you should be able to say about it.

Checkpoint 1: the cluster (Member 2)

```
$ kafka-broker-api-versions.sh --bootstrap-server $BS | grep id:
10.31.88.161:9092 (id: 2 rack: null) -> (
10.31.88.79:9092 (id: 1 rack: null) -> (
$ kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
ClusterId:        kdqk_6qZRuK8LAQGZCVNGg
LeaderId:         1
CurrentVoters:    [{"id": 1, "endpoints": ["CONTROLLER://10.31.88.79:9093"]}]
CurrentObservers: [{"id": 2, ...}]
```

- **api-versions** lists every broker the cluster knows, with the address it *advertises* to clients. Two lines with ZeroTier IPs = both brokers are registered and reachable.
- **ClusterId**: the ID created once on VM2 (`kafka-storage.sh random-uuid`). Every node's storage is formatted with it; a node with a different ID can't join.
- **LeaderId: 1**: the active controller is node 1, i.e. VM2. **CurrentVoters** lists the nodes that vote in the controller election: only node 1.
- **CurrentObservers**: nodes that follow the controller's metadata log without voting. Broker 2 (VM3) is one.
- **--list** shows `orders` and `order_metrics` (plus Kafka's internal `__consumer_offsets` once a consumer group has been used).

Checkpoint 2: raw orders VM1 -> Kafka -> VM4

```
(VM2)  kafka-console-consumer.sh ... --topic orders --group ta-observer-teamXX-run1 \
         --consumer-property auto.offset.reset=latest --max-messages 3 --timeout-ms 30000
(VM1)  python producer/producer1.py --input order.jsonl
       published 62 records ...   partition 0: 31   partition 1: 31   (split varies)
(VM2)  {...3 raw order lines...}   Processed a total of 3 messages
(VM4)  python consumer/consumer1.py ...   landed 62 records
(VM4)  wc -l outputs/orders_landed.jsonl  ->  62
```

- **The observer** is the TA's own consumer. `auto.offset.reset=latest` makes a brand-new group start at the *end* of the topic, so it only sees messages sent after it started. That's why it must start before Producer 1. `--max-messages 3` stops after 3; `--timeout-ms 30000` gives up after 30 s of silence.
- **62 published, not 48 or 44**: Producer 1 sends every line unchanged, junk included. Cleaning is Spark Job 1's job.
- **The split over partitions varies** because Producer 1 sends with no key, so each message goes to a randomly chosen partition.
- **62 landed**: Consumer 1 copies the whole topic (all partitions, from offset 0) into a file. `wc -l` counts lines: one message = one line.

Checkpoint 3: enriched orders VM4 -> Kafka -> VM1

```
(VM4) python spark/spark_job1.py ...
  dropped by cleaning rules:
     distance_km<=0                          1
     invalid_timestamp:accepted_timestamp    1
     invalid_timestamp:delivered_timestamp   1
     missing_or_invalid:city                 1
     missing_or_invalid:order_id             2
     missing_or_invalid:restaurant_id        1
     order_amount<=0                         1
     status_not_delivered                    6
```

```
  valid records: 48   exact duplicates removed: 4   enriched records written: 44
(VM4) python producer/producer2.py ...  published 44 records  (16 / 28)
(VM1) python consumer/consumer2.py ...  landed 44 records
(VM1) python spark/spark_job2.py ...    6 rows -> restaurant_summary.csv
                                        4 rows -> city_summary.csv
```

- **The drop report** lists the first rule each bad record breaks. 14 dropped, 48 valid, 4 exact duplicates removed, 44 written. Part 4 explains every line.
- **16 / 28 every time**: Producer 2 sends with the key `order_id`. Kafka hashes the key (murmur2) to pick the partition, so the same order always lands in the same partition.
- **Consumer 2 lands exactly the same 44 records**: Kafka delivered every record unchanged. Sorted, the file is identical to `order_metrics.jsonl`.
- **The CSVs** are what the portal grades for accuracy. Their numbers are in Part 4 and Part 7.

Checkpoint 4: broker failure and recovery (Members 2, 1, 3)

```
Before:  orders  Partition: 0  Leader: 2  Replicas: 2,1  Isr: 2,1
         orders  Partition: 1  Leader: 1  Replicas: 1,2  Isr: 1,2
(VM1)    producer1.py --file order.jsonl --delay 0.5   ->  sent 1/62 ... sent 4/62 ...
(VM3)    pkill -9 -f kafka.Kafka
After:   orders  Partition: 0  Leader: 1  Replicas: 2,1  Isr: 1     <- leader moved
         orders  Partition: 1  Leader: 1  Replicas: 1,2  Isr: 1
(VM1)    ... short pause ... sent 62/62   published 62 records       <- no data lost
(VM3)    restart broker 2  ->  Isr: 1,2 again, but every Leader is still 1
(VM2)    kafka-leader-election.sh --election-type preferred ...
         orders  Partition: 0  Leader: 2  ...  Isr: 1,2          <- leadership restored
```

Column	Meaning	Why it changes (or doesn't)
Leader	The broker currently serving reads and writes for that partition	Moves from 2 to 1 when broker 2 dies; moves back after the preferred-leader election
Replicas	Where copies are *assigned* to live; the first one is the **preferred** leader	Never changes during a failure: it's the plan, not the current state
Isr	In-sync replicas: copies that are fully caught up	Shrinks to 1 while broker 2 is down; grows back once it catches up
Elr / LastKnownElr	Eligible leader replicas (a newer Kafka feature, KIP-966)	Empty in our runs; not part of the assignment

- **Why the producer keeps going**: it uses `acks=all` (wait for every *in-sync* copy, now just broker 1) and the brokers allow `min.insync.replicas=1`. During the ~10 s while the leader moves, sends fail and the producer retries (30 tries, 1 s apart), so no record is lost.
- **124 messages** are now in `orders`: 62 from checkpoint 2 + 62 from the slow producer. Kafka keeps every message.
- **Why the leader doesn't move back by itself straight away**: Kafka only moves leadership back to the preferred replica on a timer (`auto.leader.rebalance.enable`, checked every few minutes). The election command forces it immediately.

Part 3. Member sections

Each section explains that member's code and role, then lists the questions most likely to be asked about it. Read all four, and know yours in depth.

Member 1 (VM1): Producer 1, Consumer 2, Spark Job 2

producer1.py + publish.py. Reads `order.jsonl` line by line and sends each line, unchanged, to topic `orders`. Settings: `acks='all'`, `retries=30`, `retry_backoff_ms=1000`, `linger_ms=10`. It waits for every send to be confirmed, so 'published 62' means 62 *acknowledged*. With `--delay 0.5` it sends one record every half second and prints each confirmation (used in checkpoint 4).

consumer2.py + landing.py. Copies the whole topic `order_metrics` into `order_metrics_landed.jsonl`: no consumer group (`group_id=None`), `assign()` all partitions, `seek_to_beginning`, write each message as a line, and stop once every partition is read up to its end offset and nothing new arrived for 10 s. The file is overwritten, so a rerun gives the same file.

spark_job2.py. Reads the landed file with a fixed schema (`DROPMALFORMED`), removes exact duplicates over all 15 columns, caches it, then `groupBy` restaurant and `groupBy` city with `count`, `avg` and a conditional count of DELAYED orders; `delay_rate_pct = 100 x delayed / count`; everything rounded to 2 decimals; rows sorted; collected and written with Python's `csv` module as one file each.

Q. Why does Producer 1 send the dirty records too?

The producer is a transport, not a validator. The spec makes Spark Job 1 responsible for cleaning; if the producer also filtered, the rules would be split across two places.

Q. Why no message key in Producer 1?

To get a key (order_id) it would have to parse the JSON, and some lines aren't valid orders. Without a key, messages are spread over both partitions.

Q. What does acks=all mean? Alternatives?

The leader confirms a write only after every in-sync replica has it. acks=0 = fire and forget; acks=1 = leader only (lost if the leader dies before the follower copies it). acks=all is the safest.

Q. Why didn't the slow producer fail when VM3 was killed?

acks=all only waits for the in-sync replicas, which shrink to broker 1; min.insync.replicas=1 allows that. While the leader moved, sends failed briefly and were retried (30 retries, 1 s apart).

Q. Why doesn't your consumer use a consumer group?

A group remembers committed offsets and shares partitions between members, so a second run would read nothing. Ours is a batch landing step that must copy the entire topic every time, so it assigns all partitions to itself and starts at offset 0.

Q. How does the consumer know when to stop?

Kafka consumers normally run forever. Ours stops when its position equals the end offset of every partition and no message has arrived for 10 s (`--idle-timeout`). `--wait-timeout` (120 s) lets it start before the producer.

Q. Why dropDuplicates in Job 2 when it removes 0?

The spec says both jobs count exact duplicates once. It also protects against a producer being re-run: then Consumer 2 would land 88 lines and the CSVs would still be right.

Q. Why collect() and write with the csv module instead of df.write.csv()?

Spark's writer creates a folder of part files; the spec wants one file with an exact name. The results are tiny (6 and 4 rows), so collecting them to the driver is safe.

Q. Why .cache() in Job 2?

The deduplicated data is used by three actions (a count and two aggregations). Without caching, Spark would re-read and re-deduplicate the file for each one.

Q. How is delay_rate_pct computed and rounded?

From the exact counts: 100 x delayed / orders, then rounded once to 2 decimals. Example R05: 100 x 3 / 7 = 42.857 -> 42.86.

Member 2 (VM2): broker 1 + KRaft controller

Runs Kafka with `process.roles=broker,controller` and `node.id=1`. It listens on **9092** (clients and the other broker) and **9093** (controller traffic). It made the cluster ID, formatted its storage, created both topics, and in the demo runs the TA's cluster checks, both observers, the describe commands and the leader election.

Setting	Value / meaning
`process.roles`	broker,controller: this node is both a broker and the controller ('combined mode')
`node.id`	1 (VM3 is 2); the number in Leader / Replicas / Isr
`controller.quorum.voters`	`1@10.31.88.79:9093`: the only voter, so the only possible controller
`listeners`	`PLAINTEXT://0.0.0.0:9092, CONTROLLER://0.0.0.0:9093`: listen on every network interface
`advertised.listeners`	`PLAINTEXT://10.31.88.79:9092`: the address told to clients; must be the ZeroTier IP
`min.insync.replicas`	1: a write with acks=all succeeds with just one in-sync copy
`auto.create.topics.enable`	false: topics are only created on purpose, with 2 partitions and RF 2
`offsets.topic.replication.factor`	2: Kafka's internal offsets topic is also copied to both brokers

Q. What is KRaft?

Kafka's built-in consensus mode (Kafka + Raft). The cluster's metadata (brokers, topics, partition leaders, ISR) is kept in an internal log managed by controller nodes using the Raft protocol. Before KRaft, Kafka needed a separate ZooKeeper cluster for this. Kafka 3.9 still allows ZooKeeper (it's the last version that does); Kafka 4 is KRaft-only.

Q. What does the controller do?

It's the cluster's brain: tracks which brokers are alive through heartbeats, elects partition leaders, updates the ISR, and records topic creation. When VM3 stopped sending heartbeats, the controller fenced it and made broker 1 the leader everywhere.

Q. Why must VM2 never be stopped?

It's the only controller voter. Without it there is no controller, so nobody can elect leaders or change metadata, and its broker holds a copy of every partition. A highly available setup needs 3 controllers so a majority (2) survives one failure.

Q. listeners vs advertised.listeners?

listeners is where the broker binds (0.0.0.0 = every interface). advertised.listeners is the address it puts in metadata for clients. A client contacts any bootstrap server, gets this metadata, then connects to each leader at its advertised address, so it must be reachable from the other VMs (the ZeroTier IP). With 'localhost' advertised, remote clients connect to themselves and fail.

Q. Why does VM3 need VM2's cluster ID?

Every node's storage is formatted with a cluster ID, and nodes only join a cluster with the same ID. If VM3 made its own, it would refuse to join.

Q. What does the preferred leader election do?

Each partition's preferred leader is the first broker in its Replicas list. After VM3 comes back, broker 1 still leads everything, so the load is uneven. The election moves leadership back to the preferred brokers right away. It fails with 'PreferredLeaderNotAvailable' if the preferred broker isn't back in the ISR yet.

Q. Why ports 9092 and 9093 on VM2, but only 9092 on VM3?

9093 is the controller listener, and only VM2 runs a controller. VM3 connects out to VM2:9093; nothing connects to a controller on VM3.

Q. Why is offsets.topic.replication.factor 2?

Kafka stores consumer-group offsets in the internal `__consumer_offsets` topic. The TA's observers use groups, so that topic must survive VM3 failing too.

Member 3 (VM3): broker 2

Runs Kafka with `process.roles=broker` and `node.id=2` only. It joins VM2's cluster (same cluster ID, `controller.quorum.voters` pointing at VM2:9093), holds the second copy of every partition, and in checkpoint 4 is killed with `pkill -9` and then restarted.

Q. What happens, step by step, when VM3 is killed?

1) Broker 2 stops sending heartbeats to the controller on VM2. 2) After the session timeout (about 9 s) the controller fences it and removes it from the ISR of every partition. 3) Where broker 2 was leader, the controller makes broker 1 (still in the ISR) the leader. 4) Clients refresh their metadata and talk to broker 1. Because RF = 2, broker 1 already has a full copy: no data lost, every partition stays available.

Q. Why kill -9 rather than a normal stop?

`pkill -9` kills the process instantly, like a crash or power cut, so the broker can't hand over its partitions politely. That's the realistic failure the test is about.

Q. What happens when VM3 comes back?

It registers with the controller again, its replicas fetch the messages they missed from the leaders, and once caught up they rejoin the ISR (Isr back to 1,2). Leadership stays on broker 1 until the preferred-leader election (or Kafka's periodic rebalance).

Q. Why is VM3 an 'observer' in the quorum output?

It isn't a controller voter; it only follows the controller's metadata log to stay up to date. Only node 1 votes.

Q. What if the replication factor were 1?

Each partition would exist on one broker only. Killing VM3 would make the partitions it held unavailable, and consumers would miss that data. RF 2 is exactly what protects against this.

Q. Why do both brokers need the same Kafka version?

Brokers share one metadata format and protocol. We hit this ourselves: with different versions, broker 2 connected to the controller but was never accepted ('waiting for the controller to acknowledge that we are caught up'). Everyone now runs 3.9.2, the TA's version.

Q. What is initial.broker.registration.timeout.ms?

How long a starting broker waits to register and catch up with the controller before giving up. We raised it to 5 minutes for broker 2 because our ZeroTier link between the VMs can be slow.

Member 4 (VM4): Consumer 1, Spark Job 1, Producer 2

consumer1.py: the same landing code as Consumer 2, for topic `orders` -> `orders_landed.jsonl` (62 lines).

spark_job1.py: reads the landed file with an explicit schema in **PERMISSIVE** mode (bad JSON goes to `_corrupt_record` instead of crashing), parses timestamps with `try_to_timestamp` (null on bad input), labels each record with the first rule it breaks, keeps the valid ones, removes exact duplicates over all 14 input fields, computes the metrics, rounds them, sorts by order_id and writes one JSON object per line. The session time zone is UTC.

producer2.py: sends the 44 enriched lines to `order_metrics`, **keyed by order_id**.

Metric	Formula (all in minutes, from full timestamps)
preparation_time_min	picked_up - accepted
delivery_time_min	delivered - picked_up
fulfillment_time_min	delivered - order_timestamp

Metric	Formula (all in minutes, from full timestamps)
delivery_speed_kmph	distance_km / (delivery_time_min / 60), via try_divide (null instead of a crash on 0)
delay_min	max(0, delivered - expected)
delivery_status	ON_TIME if delay_min = 0, else DELAYED
delay_category	0 ON_TIME; up to 15 SLIGHT_DELAY; up to 30 MODERATE_DELAY; over 30 SEVERE_DELAY

Q. Which cleaning rules does Job 1 apply, in what order?

Malformed JSON; then a missing or blank string field (order_id, customer_id, restaurant_id, delivery_partner_id, city, status); then a missing number; then an invalid timestamp; then distance_km <= 0; then order_amount <= 0; then status not DELIVERED. Each record is labelled with the *first* rule it breaks.

Q. Why clean first and then remove duplicates?

Invalid copies are thrown out by the rules anyway, so deduplication only has to compare valid records. On our data the result is the same either way, but it's the logical order.

Q. Why not deduplicate by order_id?

The spec says *exact* duplicates. Two records with the same order_id but different values are different data, and picking one would invent a business rule. After cleaning, all 44 order_ids are unique anyway.

Q. How do you handle orders that cross midnight?

Durations come from full timestamps (date + time) converted to seconds, not from the time of day. T040023: picked up 23:20, delivered 00:02 the next day = 42 min, not -1398.

Q. Why set the Spark time zone to UTC?

The timestamps have no zone. In a zone with daylight saving, a duration across the clock change would be off by an hour; UTC has none.

Q. What is PERMISSIVE mode and _corrupt_record?

PERMISSIVE keeps every line; if one can't be parsed, its fields become null and the raw text goes into _corrupt_record, so we can count and drop it ourselves. DROPMALFORMED silently drops such lines (used in Job 2, whose input is already clean). FAILFAST would stop the job.

Q. Why is Producer 2 keyed by order_id, and why always 16 / 28?

Its input is guaranteed-valid JSON, so it can read the key. Kafka hashes the key (murmur2) modulo the number of partitions, so the same order always goes to the same partition: deterministic 16 / 28.

Q. 15.5 minutes of delay: SLIGHT or MODERATE?

The spec's ranges assume whole minutes. We use >0 to 15 = SLIGHT, >15 to 30 = MODERATE, >30 = SEVERE, which matches for whole minutes and leaves no gaps. All our delays are whole minutes (8, 22, 26, 36, 42).

Part 4. Questions about the outputs

The cleaning funnel: 62 -> 48 -> 44

Reason (Spark label)	Count	Which records
status_not_delivered	6	T04X0001-T04X0006: CANCELLED, PENDING, CANCELLED, FAILED, CANCELLED, REFUNDED
missing_or_invalid:order_id	2	two {"record_type":"invalid_order_record",...} lines: valid JSON, not orders
missing_or_invalid:restaurant_id	1	{"order_id":"BROKEN","customer_id":"C"}: first missing field checked
missing_or_invalid:city	1	a T040002 copy with no city
invalid_timestamp:accepted_timestamp	1	a T040005 copy with accepted_timestamp = "bad"
invalid_timestamp:delivered_timestamp	1	a T040003 copy with delivered_timestamp = "not-a-timestamp"
distance_km<=0	1	a T040001 copy with distance_km = -2
order_amount<=0	1	a T040004 copy with order_amount = 0
Total dropped	**14**	62 - 14 = **48 valid**
Exact duplicates	4	T040006, T040017, T040028, T040039 each appear twice -> 48 - 4 = **44**

Q. Why is malformed_json 0, when there are 'broken' records?

The BROKEN and invalid_order_record lines are syntactically valid JSON, so Spark parses them; they fail the missing-field rule instead. The malformed rule is still there for lines that aren't JSON at all.

Q. T040001-T040005 are in the output, but their bad copies were dropped. How?

Each of them also has a valid copy elsewhere in the file. Only the bad copy breaks a rule; the valid one is kept.

Q. Why does orders_landed.jsonl have 62 lines but in a different order from order.jsonl?

It's a copy of the topic, read partition by partition. The two partitions interleave, so the order differs; the set of lines is the same.

Q. Why do order_metrics.jsonl and order_metrics_landed.jsonl contain the same 44 records?

Kafka delivers every record unchanged. Sorted, the two files are byte-identical (same SHA-256).

Worked example: T040023 (crosses midnight)

```
order 23:06   accepted 23:08   picked_up 23:20   expected 23:40   delivered 00:02 (next day)
distance 2.6 km   amount 858
preparation_time_min = 23:20 - 23:08              = 12.0
delivery_time_min    = 00:02 (+1 day) - 23:20     = 42.0
fulfillment_time_min = 00:02 (+1 day) - 23:06     = 56.0
delivery_speed_kmph  = 2.6 / (42 / 60) = 3.714... -> 3.71
delay_min            = max(0, 00:02 - 23:40)      = 22.0  -> DELAYED, MODERATE_DELAY
```

restaurant_summary.csv

restaurant	orders	avg amount	avg prep	avg delivery	delayed	delay %	city
R01	8	548.5	13.13	22.0	4	50.0	Bengaluru

restaurant	orders	avg amount	avg prep	avg delivery	delayed	delay %	city
R02	8	539.25	13.38	27.0	8	100.0	Bengaluru
R03	7	664.0	13.86	32.0	4	57.14	Mumbai
R04	7	639.57	12.14	37.0	7	100.0	Mumbai
R05	7	615.14	12.0	42.0	3	42.86	Pune
R06	7	712.14	13.43	47.0	7	100.0	Hyderabad

city_summary.csv

city	orders	avg amount	avg delivery	delayed	delay %
Bengaluru	16	543.88	24.5	12	75.0
Hyderabad	7	712.14	47.0	7	100.0
Mumbai	14	651.79	34.5	11	78.57
Pune	7	615.14	42.0	3	42.86

Q. How can you check the city numbers from the restaurant numbers?

Each restaurant is in exactly one city. Pune = R05 and Hyderabad = R06 exactly. Bengaluru = R01 + R02: 8 + 8 = 16 orders, 4 + 8 = 12 delayed, 12 / 16 = 75%, average delivery (22 + 27) / 2 = 24.5 (equal weights because both have 8 orders). Mumbai: (32 x 7 + 37 x 7) / 14 = 34.5.

Q. Why is avg_delivery_time_min a round number for every restaurant?

In this dataset every order of a restaurant takes the same delivery time: R01 22, R02 27, ... R06 47 minutes, 5 minutes apart. The average of identical values is that value.

Q. Why do some values show 548.5 or 22.0 instead of two decimals?

They are already exact at 2 decimals (548.50, 22.00); the CSV simply doesn't print trailing zeros.

Q. How many orders are delayed overall, and by category?

33 of 44 delayed (11 on time). Categories: ON_TIME 11, SLIGHT_DELAY 11 (8 min), MODERATE_DELAY 15 (22 and 26 min), SEVERE_DELAY 7 (36 and 42 min).

Q. Which restaurant is best and worst on delays?

R05 / Pune has the lowest delay rate, 42.86%. R02, R04, R06 and Hyderabad are at 100%. R01 is fastest (22 min average delivery), R06 slowest (47 min).

Part 5. Concept questions

Kafka basics

Q. What is Kafka, and why use it instead of copying files?

A distributed, replicated, append-only log. Producers write messages to topics; consumers read them independently. It decouples the stages (VM1 and VM4 never talk directly), stores the data durably in two copies, and lets any stage re-read it. The spec also forbids copying the file from VM4 to VM1.

Q. Topic, partition, offset?

A topic is a named stream. It's split into partitions, each an ordered log stored on brokers: the unit of parallelism and of replication. An offset is a message's position inside one partition (0, 1, 2, ...). Order is guaranteed only within a partition.

Q. What is a broker? A bootstrap server?

A broker is a Kafka server that stores partitions and serves clients. A bootstrap server is the first broker address a client contacts to discover the whole cluster; listing both brokers means a client can start even if one is down.

Q. Why 2 partitions?

The spec asks for it. Two partitions let two consumers in a group read in parallel, and let leadership be spread over both brokers (one leader each).

Q. Why replication factor 2 and not 3?

The replication factor can't exceed the number of brokers; with 2 brokers, 2 is the maximum, and each broker holds a full copy. For production: 3 brokers, RF 3, min.insync.replicas 2.

Q. What is the ISR, and why does it matter?

In-sync replicas: the copies that are fully caught up with the leader. Only an ISR member can become leader without losing data, and acks=all waits for exactly these copies.

Q. What does min.insync.replicas=1 mean? What if it were 2?

A write with acks=all succeeds with just one in-sync copy, so producing works while VM3 is down. With 2, writes would fail with NotEnoughReplicas until VM3 returned; reads would still work.

Q. What delivery guarantee does the pipeline have?

At-least-once: with retries, a record can be written twice, and re-running a producer appends the data again. That's why both Spark jobs remove exact duplicates, so the results stay correct.

Q. What is a consumer group, and what's auto.offset.reset?

A group is a set of consumers sharing a topic's partitions, with committed offsets stored in __consumer_offsets so they can resume. auto.offset.reset decides where a group with no committed offset starts: 'earliest' (beginning) or 'latest' (only new messages), which the TA's observer uses.

Q. How does a keyed message choose its partition?

The producer hashes the key with murmur2 and takes it modulo the number of partitions. Same key, same partition, so per-key order is preserved.

KRaft and the controller

Q. KRaft vs ZooKeeper?

Older Kafka stored cluster metadata in a separate ZooKeeper ensemble. KRaft stores it in an internal metadata log replicated among controller nodes with the Raft protocol, so there's one system to run. Kafka 3.9 supports both (it's the last version with ZooKeeper); Kafka 4.0 removed ZooKeeper.

Q. What is the Raft protocol in one sentence?

A consensus algorithm where nodes elect a leader by majority vote; the leader appends entries to a log and an entry is committed once a majority has it. With 1 voter, that one node is always the majority.

Q. Why is one controller a single point of failure, and how would you fix it?

If the only voter dies, there's no majority, so no leader elections or metadata changes. Use 3 (or 5) controller voters so a majority survives one (or two) failures.

Q. What does 'fenced' mean?

When a broker misses heartbeats beyond the session timeout, the controller fences it: it's treated as dead, removed from the ISR and can't lead partitions until it registers again.

Spark

Q. Why Spark for only 62 records?

The assignment is about the architecture. The same code runs unchanged on millions of records or on a cluster (change SPARK_MASTER from local[*]). Spark gives declarative, distributed transformations.

Q. What does local[*] mean?

Spark runs in one JVM on this machine, with one worker thread per CPU core; no cluster manager is needed.

Q. Transformations vs actions; lazy evaluation?

Transformations (filter, withColumn, groupBy, dropDuplicates) only build a plan. Nothing runs until an action (count, collect). Spark's Catalyst optimiser then optimises and runs the whole plan. That's why we cache DataFrames used by several actions.

Q. Why an explicit schema instead of inferSchema?

Inference reads the file twice and guesses types (a bad value could turn a number column into a string). With an explicit schema the types are guaranteed, and mismatches become corrupt records.

Q. Why doesn't Spark read Kafka directly (Structured Streaming)?

The spec forbids it: Kafka data must first be landed to a local file. That keeps the stages decoupled and the Spark input inspectable and reproducible.

Q. How does dropDuplicates work?

Given a list of columns, it keeps one row per distinct combination of them. We pass all the columns, so only rows identical in every field collapse. Internally it's a shuffle + aggregation.

Q. How does Spark round, and why does the verifier use Decimal?

Spark's round is HALF_UP (2.345 -> 2.35). Python's built-in round is banker's rounding, which can differ. The verifier (tests/verify_outputs.py) uses Decimal with ROUND_HALF_UP so both agree exactly.

Networking

Q. What is ZeroTier for?

A virtual private LAN over the internet. Each VM gets an IP on the same network (10.31.88.x), so brokers and clients on different home networks reach each other without port forwarding.

Q. Why does ZeroTier have to run inside the VM, not just on the host?

Kafka runs inside the VM. If ZeroTier is only on the host, connections to the host's ZeroTier IP aren't passed into the VM, so nobody can reach the broker. (VM1 and VM4 only connect out, so for them the host is enough.)

Q. What's the difference between 'Connection refused' and 'timed out'?

Refused: the machine answered, but nothing listens on that port (broker not running). Timed out: no answer at all, usually a firewall dropping the packets, or the machine being offline.

Part 6. Tricky and what-if questions

Q. What if VM2 were stopped instead of VM3?

The only controller would be gone: no leader elections, no metadata changes. Partitions led by broker 1 would become unavailable, and a restarted broker 2 couldn't register. The cluster can't fail over. That's why the spec says to stop VM3, and why controller handover is out of scope.

Q. What if Producer 1 is run twice?

`orders` holds 124 messages and Consumer 1 lands 124 lines, but Spark Job 1 still outputs 44, because the extra copies are exact duplicates. For a clean run, delete and recreate the topics.

Q. What if both brokers were killed?

No broker holds the data, so producers and consumers fail until a broker returns. The data isn't lost: it's on each broker's disk and is served again after the restart.

Q. What if VM3 dies while the slow producer is mid-send?

Exactly the demo: sends to partitions led by broker 2 fail for ~10 s until broker 1 becomes leader, and the producer's retries cover the gap. All 62 arrive.

Q. Could a record be lost? Under what settings?

With acks=1, if the leader died before the follower copied a record. With unclean leader election enabled, an out-of-sync replica could become leader and lose data. We use acks=all and Kafka's default (unclean.leader.election.enable=false).

Q. Why does the TA's observer use a new group name each run (run1, run2)?

A new group has no committed offsets, so auto.offset.reset=latest applies and it starts at the end. Reusing a group would resume from where it stopped last time.

Q. Why does Replicas still show 2 when broker 2 is dead?

Replicas is the assignment (the plan), not the live state. The live state is Leader and Isr.

Q. Why does the first broker in Replicas matter?

It's the preferred leader. Kafka spreads preferred leaders across brokers so the load is balanced, and the preferred-leader election moves leadership back to it.

Q. Your outputs are 'not hardcoded'. How can you prove it?

Every number is computed from the input file by the code; tests/verify_outputs.py recomputes everything in plain Python from order.jsonl and compares field by field ('OK: 44 enriched records ...'). A different dataset gives different outputs.

Q. What would you change for production?

3 dedicated controllers; 3 brokers with RF 3 and min.insync.replicas 2; SSL/SASL instead of PLAINTEXT; an idempotent or transactional producer for exactly-once; Spark on a real cluster writing to distributed storage.

Q. Why kafka-python and not confluent-kafka?

Pure Python, installs with pip with no native library, behaves the same on Windows, WSL and Linux. Performance doesn't matter for 62 records.

Q. What problems did you hit, and what did you learn?

Brokers advertising 'localhost' (other VMs couldn't connect), a firewall dropping Kafka ports, ZeroTier needing to run inside the VM, and mismatched Kafka versions stopping broker 2 from joining (fixed by everyone using 3.9.2). Lesson: the network and version setup matter as much as the code.

Part 7. Numbers to remember

Item	Value
Raw lines	62
Dropped by cleaning	14: 6 status, 2 no order_id, 1 BROKEN, 1 no city, 2 bad timestamps, 1 distance, 1 amount
Valid / duplicates removed / enriched	48 / 4 (T040006, T040017, T040028, T040039) / 44
orders partition split	random each run, e.g. 31 / 31
order_metrics partition split	always 16 / 28 (keyed by order_id)
Restaurants / cities	6 (R01-R06) / 4 (Bengaluru, Hyderabad, Mumbai, Pune)
Delayed orders	33 of 44 (11 on time)
Delay categories	ON_TIME 11, SLIGHT 11, MODERATE 15, SEVERE 7
Highest / lowest delay rate	100%: R02, R04, R06, Hyderabad / 42.86%: R05, Pune
Fastest / slowest delivery	R01 22.0 min / R06 47.0 min
After the slow producer	orders holds 124 messages (62 + 62)
Topics	orders, order_metrics: 2 partitions, RF 2, min.insync.replicas 1
Nodes and ports	VM2 = node 1 (9092 broker, 9093 controller); VM3 = node 2 (9092)
Producer settings	acks=all, retries=30, retry_backoff_ms=1000, linger_ms=10
Consumer stop rule	caught up on every partition + 10 s idle (wait up to 120 s for the first record)
Versions	Kafka 3.9.2 (KRaft) on every broker, PySpark 4.2.0, kafka-python 2.2.15, Java 17+